

Chapter 8: Lifecycle Hooks

1. Overview

Every Angular directive and component has a lifecycle managed by Angular itself: created, rendered, checked for changes repeatedly during change detection, and eventually destroyed. Angular exposes this lifecycle through a fixed set of interfaces/methods — the **lifecycle hooks** — that let you tap into specific moments to run initialization logic, react to input changes, inspect the rendered DOM, or clean up resources.

Understanding lifecycle hooks is not optional trivia — it is the backbone of correctly written Angular components. Getting the order wrong, or doing the wrong kind of work in the wrong hook, is the single most common source of bugs like "ExpressionChangedAfterItHasBeenCheckedError," memory leaks from unsubscribed observables, and components that read stale `@ViewChild` / `@ContentChild` references.

This chapter covers all eight hooks, the constructor's special (non-)role, the exact firing order in a parent/child hierarchy, how Ivy actually invokes these hooks during change detection, and the classic interview traps.

2. Core Concepts

2.1 The Full List of Hooks

Hook	Interface	Fires when	Fires how often
<code>ngOnChanges</code>	<code>OnChanges</code>	Before <code>ngOnInit</code> , and whenever a bound <code>@Input()</code> property changes	Every CD cycle where an input reference changes (0+ times)
<code>ngOnInit</code>	<code>OnInit</code>	After the first <code>ngOnChanges</code> (or immediately if no inputs)	Once
<code>ngDoCheck</code>	<code>DoCheck</code>	On every change detection run, custom check	Every CD cycle
<code>ngAfterContentInit</code>	<code>AfterContentInit</code>	After content (<code><ng-content></code> projected children) has been initialized	Once
<code>ngAfterContentChecked</code>	<code>AfterContentChecked</code>	After every check of projected content	Every CD cycle
<code>ngAfterViewInit</code>	<code>AfterViewInit</code>	After the component's own view (and child views) has been initialized	Once
<code>ngAfterViewChecked</code>	<code>AfterViewChecked</code>	After every check of the component's view (and child views)	Every CD cycle
<code>ngOnDestroy</code>	<code>OnDestroy</code>	Just before Angular destroys the directive/component	Once

The **constructor** is not a lifecycle hook at all — it is plain TypeScript/JavaScript class construction, invoked when Angular's injector instantiates the class via dependency injection.

2.2 Firing Order for a Single Component (First Render)

```
constructor
  -> ngOnChanges      (only if it has @Input()-bound properties, and only on first change)
  -> ngOnInit
  -> ngDoCheck
  -> ngAfterContentInit
  -> ngAfterContentChecked
  -> ngAfterViewInit
  -> ngAfterViewChecked
```

On **every subsequent change detection cycle** (e.g., triggered by an event, timer, HTTP response, etc.), the following subset repeats, in this order:

```
ngOnChanges      (only if bound inputs changed)
  -> ngDoCheck
  -> ngAfterContentChecked  (ngAfterContentInit does NOT run again)
  -> ngAfterViewChecked    (ngAfterViewInit does NOT run again)
```

Finally, exactly once, when the component is removed from the DOM (via `*ngIf`, router navigation away, `viewContainerRef.clear()`, etc.):

```
ngOnDestroy
```

2.3 Why the Constructor Should Stay Lean

The constructor's only job in Angular is **dependency injection** — declaring and receiving injected services:

```
constructor(private http: HttpClient, private route: ActivatedRoute) {}
```

Reasons to avoid real work in the constructor:

1. **@Input() properties are not yet bound.** Angular assigns input bindings *after* construction, right before calling `ngOnChanges` / `ngOnInit`. Reading `this.someInput` in the constructor gives `undefined`.
2. **The view/template is not yet initialized.** `@ViewChild` references are `undefined` in the constructor.
3. **Testability.** TestBed creates the component via the constructor during `createComponent()`, but `fixture.detectChanges()` is what triggers `ngOnInit`. Heavy constructor logic makes it harder to control test setup precisely (you cannot "pause" between construction and initialization logic).
4. **Separation of concerns.** DI wiring (constructor) vs. initialization logic (`ngOnInit`) is a deliberate Angular convention — mixing them makes classes harder to reason about and refactor.
5. **Exception safety during DI resolution.** If DI itself is mid-resolution (especially with circular deps or `forwardRef`), running business logic in the constructor can interact badly with injector internals.

Rule of thumb: constructor = assign injected dependencies to `private` / `readonly` fields only. All real initialization (fetching data, subscribing, reading inputs) belongs in `ngOnInit`.

2.4 ngOnChanges in Depth

- Fires **only** for properties decorated with `@Input()` (including inputs bound via `[prop]="expr"` or the `inputs: [...]` array in `@Component`/`@Directive` metadata, and Angular 17.1+ signal inputs go through a related-but-different notification path — see Edge Cases).
- Fires **before** `ngOnInit` on the first pass.
- Receives a `SimpleChanges` object: a dictionary keyed by input property name, where each value is a `SimpleChange` with `previousValue`, `currentValue`, and `firstChange: boolean`.
- Angular uses **reference equality** (not deep equality) to decide whether an input "changed." Mutating an object/array in place without replacing the reference will **not** trigger `ngOnChanges`.
- If a component has zero `@Input()`-bound properties, `ngOnChanges` never fires at all — not even once.

```
ngOnChanges(changes: SimpleChanges): void {
  if (changes['userId']) {
    const { previousValue, currentValue, firstChange } = changes['userId'];
    console.log({ previousValue, currentValue, firstChange });
  }
}
```

2.5 ngOnInit

- Runs exactly once, after the first `ngOnChanges` call (or immediately after construction if there are no data-bound inputs).
- The correct place for: initial data fetching, subscribing to observables, setting up initial component state that depends on `@Input()` values.
- Guaranteed inputs are populated here — unlike the constructor.

2.6 ngDoCheck — Custom Change Detection

- Fires on **every** change detection cycle, immediately after `ngOnChanges` (if it ran) and immediately before `ngAfterContentChecked`.
- Purpose: implement your own dirty-checking logic for changes Angular's default binding mechanism cannot detect — e.g., mutation of an object/array in place (since Angular only compares by reference for `@Input()`s).
- **Danger:** because it runs on every single CD cycle (possibly hundreds of times per second under heavy interaction), any nontrivial logic here can tank performance. Keep it extremely cheap, or use it only for diagnostics.

```
ngDoCheck(): void {
  if (this.items.length !== this.previousLength) {
    console.log('Items array mutated in place');
    this.previousLength = this.items.length;
  }
}
```

2.7 Content vs. View Hooks

This is the most confused pair of concepts in the lifecycle system.

- **Content** = elements **projected into** a component from its parent via `<ng-content>` (i.e., `@ContentChild` / `@ContentChildren`). "Content" belongs conceptually to the *consumer* of the component.
- **View** = the component's **own template**, including child components it declares in its own template (i.e., `@ViewChild` / `@ViewChildren`). "View" belongs to the component itself.

```
<!-- parent.component.html -->
<app-card>
  <p #projected>I am projected content</p>
</app-card>
```

```
// card.component.ts
@Component({
  selector: 'app-card',
  template: `
    <div class="card">
      <ng-content></ng-content>
      <app-footer #viewChild></app-footer>
    </div>
  `
})
export class CardComponent implements AfterContentInit, AfterViewInit {
  @ContentChild('projected') projectedContent!: ElementRef;
  @ViewChild('viewChild') viewChild!: FooterComponent;

  ngAfterContentInit() {
    // projectedContent is now available (it comes from the PARENT)
  }
  ngAfterViewInit() {
    // viewChild is now available (it's part of CardComponent's own template)
  }
}
```

- `ngAfterContentInit` / `ngAfterContentChecked` fire **before** `ngAfterViewInit` / `ngAfterViewChecked`, because Angular must finish resolving projected content before it fully finishes checking the component's own view (the view includes the `<ng-content>` slot).
- `@ContentChild` accessed in `ngOnInit` is `undefined`; it is only guaranteed set in `ngAfterContentInit`.
- `@ViewChild` accessed in `ngOnInit` / `ngAfterContentInit` is `undefined`; it is only guaranteed set in `ngAfterViewInit`.

2.8 ngAfterViewChecked and the "Checked After It Has Been Checked" Trap

Because `ngAfterViewChecked` (and `ngAfterContentChecked`, `ngDoCheck`) run **after** Angular has already checked and rendered the current cycle, mutating a property that is bound in the template **inside these hooks** can cause:

```
ExpressionChangedAfterItHasBeenCheckedError
```

This happens because Angular (in dev mode) re-verifies bindings after CD completes to catch exactly this kind of bug — you changed a value the template depends on *after* the template was already rendered for this pass.

2.9 ngOnDestroy

- Fires once, synchronously, right before Angular tears down the component/directive and removes it from the DOM.
- The only correct place to: unsubscribe from manually-created `Subscription`s, clear `setInterval` / `setTimeout` timers, detach event listeners added via `renderer.listen` (if not using the returned teardown function), disconnect `ResizeObserver` / `IntersectionObserver`, complete `Subject`s used for the `takeUntil` pattern.
- Not guaranteed to run if the entire application is destroyed abruptly (e.g., browser tab closed) — it is an Angular-managed teardown, not a browser `beforeunload` event.

2.10 Full Parent/Child Ordering (the Interview Favorite)

For a `ParentComponent` containing one `ChildComponent` in its template, on **initial render**, the *exact* interleaved order is:

```
Parent: constructor
Parent: ngOnChanges          (if parent itself is a child of something with inputs bound)
Parent: ngOnInit
Parent: ngDoCheck
Child:  constructor
Child:  ngOnChanges
Child:  ngOnInit
Child:  ngDoCheck
Child:  ngAfterContentInit
Child:  ngAfterContentChecked
Child:  ngAfterViewInit
Child:  ngAfterViewChecked
Parent: ngAfterContentInit
Parent: ngAfterContentChecked
Parent: ngAfterViewInit
Parent: ngAfterViewChecked
```

Key rule to memorize: *"Init/Checked bubble up from children before the parent finishes its own AfterView phase, but construction and OnInit/DoCheck cascade top-down."* In other words:

- **Top-down:** constructor → ngOnChanges → ngOnInit → ngDoCheck (parent runs these, *then*

creates/initializes children, who run their own constructor → ngOnChanges → ngOnInit → ngDoCheck).

- **Bottom-up:** ngAfterContentInit/Checked and ngAfterViewInit/Checked — a component cannot claim "my view is fully checked" until all its children have finished being checked first, so children's AfterView hooks always resolve **before** the parent's AfterView hooks.

On subsequent CD cycles (no new component creation), the pattern repeats without constructor/OnInit/AfterContentInit/AfterViewInit:

```
Parent: ngOnChanges (if inputs changed) -> ngDoCheck
Child:  ngOnChanges (if inputs changed) -> ngDoCheck -> ngAfterContentChecked ->
ngAfterViewInitChecked
Parent: ngAfterContentChecked -> ngAfterViewInitChecked
```

2.11 Destroy Ordering

Destruction is **top-down** for scheduling but the child is fully destroyed (its `ngOnDestroy` runs) **before** the parent's `ngOnDestroy`:

```
Child:  ngOnDestroy
Parent: ngOnDestroy
```

This makes sense: Angular tears down the tree depth-first so children can still safely reference their (still-alive) parent context while cleaning up.

3. Code Examples

3.1 Observing the Full Order with Console Logs

```
import { Component, Input, OnChanges, OnInit, DoCheck,
        AfterContentInit, AfterContentChecked,
        AfterViewInit, AfterViewChecked, OnDestroy, SimpleChanges } from '@angular/core';

@Component({
  selector: 'app-child',
  template: `<p>Child works. value: {{ value }}</p>`
})
export class ChildComponent implements OnChanges, OnInit, DoCheck,
  AfterContentInit, AfterContentChecked, AfterViewInit, AfterViewChecked, OnDestroy {

  @Input() value!: string;

  constructor() {
    console.log('Child: constructor');
  }
  ngOnChanges(changes: SimpleChanges): void {
    console.log('Child: ngOnChanges', changes);
  }
  ngOnInit(): void {
    console.log('Child: ngOnInit');
  }
}
```

```

ngDoCheck(): void {
  console.log('Child: ngDoCheck');
}
ngAfterContentInit(): void {
  console.log('Child: ngAfterContentInit');
}
ngAfterContentChecked(): void {
  console.log('Child: ngAfterContentChecked');
}
ngAfterViewInit(): void {
  console.log('Child: ngAfterViewInit');
}
ngAfterViewChecked(): void {
  console.log('Child: ngAfterViewChecked');
}
ngOnDestroy(): void {
  console.log('Child: ngOnDestroy');
}
}

```

```

@Component({
  selector: 'app-parent',
  template: `
    <button (click)="toggle = !toggle">Toggle child</button>
    <button (click)="counter = counter + 1">Bump counter</button>
    <app-child *ngIf="toggle" [value]="counter"></app-child>
  `
})
export class ParentComponent implements OnInit, DoCheck, AfterViewInit, AfterViewChecked {
  toggle = true;
  counter = 0;

  constructor() { console.log('Parent: constructor'); }
  ngOnInit(): void { console.log('Parent: ngOnInit'); }
  ngDoCheck(): void { console.log('Parent: ngDoCheck'); }
  ngAfterViewInit(): void { console.log('Parent: ngAfterViewInit'); }
  ngAfterViewChecked(): void { console.log('Parent: ngAfterViewChecked'); }
}

```

Console output on initial load:

```
Parent: constructor
Parent: ngOnInit
Parent: ngDoCheck
Child: constructor
Child: ngOnChanges
Child: ngOnInit
Child: ngDoCheck
Child: ngAfterContentInit
Child: ngAfterContentChecked
Child: ngAfterViewInit
Child: ngAfterViewChecked
Parent: ngAfterViewInit
Parent: ngAfterViewChecked
```

Clicking "Bump counter" (updates `[value]="counter"` binding):

```
Parent: ngDoCheck
Child: ngOnChanges
Child: ngDoCheck
Child: ngAfterContentChecked
Child: ngAfterViewChecked
Parent: ngAfterViewChecked
```

Clicking "Toggle child" (removes it via `*ngIf`):

```
Child: ngOnDestroy
Parent: ngDoCheck
Parent: ngAfterViewChecked
```

3.2 Correct Subscription Cleanup Pattern

```
import { Component, OnInit, OnDestroy } from '@angular/core';
import { Subject, interval } from 'rxjs';
import { takeUntil } from 'rxjs/operators';

@Component({ selector: 'app-ticker', template: `{{ tick }}` })
export class TickerComponent implements OnInit, OnDestroy {
  tick = 0;
  private destroy$ = new Subject<void>();

  ngOnInit(): void {
    interval(1000)
      .pipe(takeUntil(this.destroy$))
      .subscribe(t => (this.tick = t));
  }

  ngOnDestroy(): void {
    this.destroy$.next();
    this.destroy$.complete();
  }
}
```



```
}
```

3.3 Detecting In-Place Mutation with ngDoCheck

```
@Component({ selector: 'app-list', template: `<li *ngFor="let i of items">{{i}}</li>` })
export class ListComponent implements OnChanges, DoCheck {
  @Input() items: number[] = [];
  private lastSeenLength = 0;

  ngOnChanges(changes: SimpleChanges): void {
    // Only fires if the PARENT rebinds items to a NEW array reference.
    console.log('Reference changed:', changes['items']);
  }

  ngDoCheck(): void {
    // Fires every cycle; catches push()/splice() on the SAME array reference.
    if (this.items.length !== this.lastSeenLength) {
      console.log('Mutation detected via ngDoCheck');
      this.lastSeenLength = this.items.length;
    }
  }
}
```

4. Internal Working

Angular's Ivy renderer does not maintain a separate "lifecycle scheduler" object per component the way View Engine's `ChangeDetectorRef` abstractions suggested conceptually — instead, hook invocation is **compiled directly into each component's generated instruction stream**.

- Each component's `@Component` decorator is compiled by the Ivy compiler into a `ComponentDef` (`@cmp`) containing, among other things, a `hostBindings` function and a **template function** with two modes: `Create` mode (runs once, allocates DOM nodes) and `update` mode (runs on every CD pass, refreshes bindings).
- Ivy tracks, per component definition, which lifecycle interfaces are implemented (`onInit`, `doCheck`, `afterContentInit`, etc.) as flags baked in at compile time (`type.prototype.ngOnInit` etc. checked via `NgOnChangesFeature`/definition features). This avoids `instanceof` checks at runtime — the compiler already knows statically what a class implements.
- Change detection walks the **logical view tree** (`LView`/`TView` data structures), not the raw DOM tree. Each `LView` corresponds to a component instance (or embedded view, e.g., from `*ngIf`/`*ngFor`). Ivy calls `refreshView()` on an `LView`, which:
 - Runs input/property bindings first (this is where `ngOnChanges` is invoked, via the `NgOnChanges` lifecycle feature wrapping `setInput`).
 - Calls `ngOnInit`/`ngDoCheck` for that view's component (`callHooks`/`executeInitAndCheckHooks` in `core/src/render3/instructions/lifecycle.ts` conceptually — hook execution is driven by a **hooks array** stored on the `TView`, alternating "init" hook flags with "check" hook flags per node index).
 - Recurses into child views/component views** — this is why content/view "checked" hooks

bubble bottom-up: the parent's `refreshView` doesn't finish (and therefore can't fire its own `AfterContentChecked` / `AfterViewChecked`) until it has recursively refreshed and thus fully processed every embedded/child view first.

4. After child view traversal is complete for a given component, Ivy fires that component's

`ngAfterContentInit` / `ngAfterContentChecked` (content projection is resolved as part of

processing `<ng-content>` slots, which happens earlier in traversal than the component's own

remaining view children) and finally `ngAfterViewInit` / `ngAfterViewChecked`.

- The **"init" hooks** (`ngOnInit`, `ngAfterContentInit`, `ngAfterViewInit`) are tracked with a per-view "init phase state" bitmask so Ivy guarantees they run exactly once even though `refreshView` itself is invoked repeatedly across the app's lifetime (every CD cycle re-enters the same function).
- `ngOnDestroy` hooks are **not** invoked during `refreshView`; they are stored in a separate `destroyHooks` / `cleanup` array on the `LView` and invoked by `removeView` / `destroyLView` when a view is detached — e.g., when `*ngIf` becomes false, a route is left, or `ViewContainerRef.remove()` / `clear()` runs. Angular walks the view being destroyed **and all its descendant views first**, invoking their destroy hooks bottom-up, then the view's own hooks — again matching the "children finish before parent" pattern, but this time for teardown.
- Change detection strategy (`Default` vs `OnPush`) affects **whether `refreshView` is entered for a given component at all** during a given root-to-leaf pass, but once entered, the internal hook-firing order described above is identical regardless of strategy. With `OnPush`, `ngDoCheck` / `ngAfterContentChecked` / `ngAfterViewChecked` simply won't run for a component on cycles where Angular determines (via marking `dirty`) that the component doesn't need checking — but `ngOnChanges` will still fire whenever a bound `@Input()` reference does change, since that itself marks the component dirty and forces a check.

5. Edge Cases & Gotchas

1. **No `@Input()` s = no `ngOnChanges` ever.** A component that only reads services/route data will never see `ngOnChanges` fire, even once. Don't rely on it as a substitute for `ngOnInit`.
2. **Reference equality, not deep equality.** `this.arr.push(x)` on an existing array bound via `[items]="arr"` will **not** trigger `ngOnChanges` in the child — the reference didn't change. Only `this.arr = [...this.arr, x]` triggers it. This trips up almost every junior dev the first time they try to react to array mutation.
3. **`SimpleChanges` keys use the *bound* input name.** If you alias an input (`@Input('userId') id: string`), the `SimpleChanges` object is keyed by `'userId'` (the public binding name), not `'id'`.
4. **`firstChange` is true on the very first call, even if `previousValue` looks meaningful.** On the first invocation `previousValue` is `undefined` regardless of what was passed—always check `changes['x'].firstChange` rather than inferring from `previousValue`.
5. **Order between multiple changed inputs in one `ngOnChanges` call is not guaranteed to match declaration order** in all Angular versions/compilation modes — never write logic that depends on which key of `SimpleChanges` "runs first"; all changed inputs arrive together in a single call, not one call per input.
6. **`ngOnChanges` fires *before* `ngOnInit`, but *after* the constructor** — so if you need the *first* input value, `ngOnInit` (not the constructor) is correct, and `ngOnChanges` also works but is arguably the wrong tool for one-time init logic (it also fires on every subsequent change).

7. **@ContentChild / @ViewChild timing.** Accessing a static `@ViewChild` in `ngOnInit` gives `undefined` — it's only populated by `ngAfterViewInit`. Using `{ static: true }` makes Angular resolve it earlier (available in `ngOnInit`), but **only works for elements that are not inside `*ngIf / *ngFor`** (i.e., not conditionally rendered), since those require a CD pass to exist at all.
8. **ExpressionChangedAfterItHasBeenCheckedError.** Mutating a template-bound property inside `ngAfterViewInit`, `ngAfterViewChecked`, or `ngAfterContentChecked` is a classic trigger, because you're changing data *after* Angular already rendered it for this tick. Common "fixes" (all trade-offs): wrap the mutation in `setTimeout(() => ..., 0)`, call `this.cdr.detectChanges()` explicitly right after, or (best) move the logic to `ngOnInit` / an async callback that naturally runs in the next CD cycle.
9. **ngOnDestroy is not called for services** unless the service is `providedIn: 'root'` / scoped to a component/module that itself gets destroyed **and** injected at a level whose injector is torn down. A singleton root-provided service is never destroyed for the app's lifetime (except in specific test/module-destroy scenarios), so relying on its `ngOnDestroy` for cleanup at the app level is meaningless.
10. **Forgetting to unsubscribe is the #1 real-world memory leak.** Any manual `.subscribe()` inside `ngOnInit` (to `interval`, a `Subject`, a `WebSocket`, `fromEvent`, a store selector, etc.) that isn't cleaned up in `ngOnDestroy` (or using `async` pipe / `takeUntilDestroyed()`) keeps the component instance alive in memory and keeps its callback executing after the component is gone — sometimes causing errors like "cannot read property of undefined" when the callback fires post-destroy and touches DOM/template state.
11. **ngDoCheck runs on every CD cycle for every instance**, including deeply nested components with `OnPush` ... actually with `OnPush` it's skipped when the component isn't marked dirty — but under `Default` strategy it runs extremely often (any event anywhere in the app can trigger a full tree CD pass). Nontrivial logic here is a well-known performance foot-gun; profile before adding real work to `ngDoCheck`.
12. **ngOnChanges vs signal-based inputs (`input()`).** With Angular's signal inputs (v17+), reading an input is done via calling the signal (`this.value()`), and reacting to changes is idiomatically done with `effect()` inside the injection context (or `computed()`), not `ngOnChanges` — `ngOnChanges` **still fires** for signal inputs (for backwards compatibility, mixed decorator/signal components), but relying on `effect()` is the modern idiomatic replacement.
13. **Destroy order with dynamically created components (`ViewContainerRef.createComponent`)** follows the same depth-first "children before parent" rule, but if you manually track a `ComponentRef` and forget to call `.destroy()` on it (e.g., you created it outside a `ViewContainerRef`-managed view, or detached it), Angular will **never** call its `ngOnDestroy` — you are responsible for destruction yourself in that case.
14. **ngAfterContentInit timing with `*ngIf` on projected content.** If the projected content itself is behind an `*ngIf` that's initially `false`, `@ContentChild` may resolve to `undefined` even inside `ngAfterContentInit`, because the projected node hasn't been created yet. You typically need `@ContentChild(..., { descendants: true })` plus a `QueryList.changes` subscription to react when it later appears.

6. Interview Questions & Answers

Q1. What is the difference between the constructor and `ngOnInit`?

A: The constructor is standard JS/TS class instantiation used exclusively for dependency injection in Angular — Angular calls it when it creates the component instance, before any bindings are resolved. `ngOnInit` is an Angular lifecycle hook called once, after Angular has set the component's initial `@Input()`-bound properties

(i.e., after the first `ngOnChanges`, if applicable). Any logic depending on input values, `@ViewChild / @ContentChild` (no — those need later hooks), or that does real initialization work (HTTP calls, subscriptions) belongs in `ngOnInit`, not the constructor.

Q2. Why does `ngOnChanges` sometimes never fire for a component?

A: `ngOnChanges` only fires for properties that are bound via `@Input()` (or the `inputs` array) *and* that are actually set through a template binding. A component with no `@Input()` properties, or one whose inputs are always read internally rather than bound by a parent, will never trigger it. It's tied specifically to Angular's property-binding mechanism, not general "the component changed."

Interviewer intent: This checks whether the candidate actually understands what "changes" `ngOnChanges` tracks (bound inputs specifically) versus assuming it's a generic "something happened" hook — a very common misconception among developers who've only skimmed docs.

Q3. What's the exact shape of the `SimpleChange` object passed to `ngOnChanges`?

A: It's a dictionary/map (`{ [propertyName: string]: SimpleChange }`) where each key is the *public* (bound) name of a changed `@Input()`, and each value is a `SimpleChange` instance with three properties: `previousValue`, `currentValue`, and a boolean `firstChange`. On the very first invocation, `firstChange` is `true` for every changed input and `previousValue` is `undefined`.

Q4. Why would mutating an array in place not trigger `ngOnChanges` in a child component?

A: Angular's default change detection (and specifically the machinery feeding `ngOnChanges`) compares the *new* value of a bound input against the *previous* value using reference equality (`!==`), not deep/structural equality. `array.push(x)` mutates the existing object without changing its reference, so from Angular's perspective "nothing changed" as far as that binding goes — `ngOnChanges` won't fire. To trigger it, you must assign a new reference, e.g. `this.arr = [...this.arr, x]`.

Q5. In a parent-child component tree, what is the exact order of hook calls on initial render?

A: Parent constructor → Parent `ngOnChanges` (if applicable) → Parent `ngOnInit` → Parent `ngDoCheck` → Child constructor → Child `ngOnChanges` → Child `ngOnInit` → Child `ngDoCheck` → Child `ngAfterContentInit` → Child `ngAfterContentChecked` → Child `ngAfterViewInit` → Child `ngAfterViewChecked` → Parent `ngAfterContentInit` → Parent `ngAfterContentChecked` → Parent `ngAfterViewInit` → Parent `ngAfterViewChecked`. The mnemonic: construction/init/doCheck cascade top-down into children, but "AfterContent"/"AfterView" phases bubble bottom-up because a parent's view/content can't be considered "checked" until every child it contains has finished being checked.

Interviewer intent: This is the single most-asked lifecycle question in Angular interviews. The interviewer wants to see if the candidate has actually run this experiment (or deeply internalized why), not just memorized a list — follow-up probes often ask "why does `AfterViewInit` bubble up instead of down?"

Q6. What's the difference between `ngAfterContentInit` and `ngAfterViewInit`?

A: `ngAfterContentInit` fires after Angular has fully initialized content **projected into** the component from its parent via `<ng-content>` — i.e., after `@ContentChild / @ContentChildren` queries are resolved. `ngAfterViewInit` fires after the component's **own template** (including any child components declared in that template, resolved via `@ViewChild / @ViewChildren`) has been fully initialized. Content belongs to the consumer of the component; view belongs to the component itself. Content hooks always fire before view hooks for a given component instance.

Q7. Why should you avoid heavy logic in the constructor?

A: Because `@Input()` bindings aren't assigned yet, the view/template doesn't exist yet (so `@ViewChild` is `undefined`), the constructor's sole documented Angular purpose is DI, and putting business logic there conflates two different lifecycle phases (instantiation vs. initialization), making the class harder to test and reason about. `ngOnInit` is guaranteed to run after inputs are set and is the idiomatic place for setup logic.

Q8. When would you use `ngDoCheck` instead of relying on `ngOnChanges`?

A: When you need to detect changes Angular's default binding comparison can't see — most commonly, in-place mutation of an object or array bound as an `@Input()` (since `ngOnChanges` only fires on reference changes). `ngDoCheck` runs on every change detection cycle regardless, so you implement your own comparison logic (e.g., caching `array.length` or a shallow clone) inside it. It's a powerful escape hatch but must be kept extremely cheap since it runs far more often than any other hook.

Q9. What happens if you set a template-bound property inside `ngAfterViewChecked`?

A: In development mode, Angular runs a secondary verification pass after change detection completes, comparing bindings against what was just rendered. If you mutate a bound value inside `ngAfterViewChecked` (or `ngAfterContentChecked` / `ngAfterViewInit`), the value the template would produce on the next check no longer matches what was already rendered this cycle, throwing `ExpressionChangedAfterItHasBeenCheckedError`. This exists specifically to catch that class of bug; production builds don't throw it, but the underlying UI inconsistency (a one-tick-stale render) still exists.

Interviewer intent: Probing whether the candidate has actually debugged this real, extremely common Angular error in production code, and understands *why* it happens (not just "wrap it in `setTimeout`" as a cargo-culted fix).

Q10. Is `ngOnDestroy` guaranteed to be called for every component?

A: It's guaranteed to be called by Angular whenever Angular itself removes the component's view from the tree — via `*ngIf` becoming false, `*ngFor` removing an item, navigating away from a route that used the component, or explicit `viewContainerRef.clear()/remove()`. It is *not* called if the browser tab/process is killed abruptly, and it will not be called for dynamically created components (via `createComponent`) that you never explicitly `.destroy()` and that aren't attached to an Angular-managed `ViewContainerRef`.

Q11. How does `onPush` change detection interact with lifecycle hooks?

A: `onPush` doesn't change the *order* of hooks or which hooks exist — it changes whether a given component is entered into a change detection pass at all. With `onPush`, Angular skips re-checking (and thus skips `ngDoCheck` / `ngAfterContentChecked` / `ngAfterViewChecked`) for a component unless it's marked dirty, which happens when: a bound `@Input()` reference changes, an event originates from within the component's template, an `async` pipe emits, or `ChangeDetectorRef.markForCheck()` / `detectChanges()` is called manually. `ngOnChanges` still fires normally whenever an input reference does change, since that's exactly the condition that marks the component dirty in the first place.

Interviewer intent: Tests whether the candidate conflates "hook order" with "change detection strategy" — a nuanced but common confusion; a strong answer separates "which hooks exist and their order" from "whether/how often a given component gets entered into a CD pass."

Q12. Why does `ngOnChanges` fire before `ngOnInit`, and can you rely on it for one-time initialization instead of `ngOnInit`?

A: `ngOnChanges` is Angular's notification that input bindings have just been set; it necessarily has to run before `ngOnInit` so that by the time `ngOnInit` executes, at least the first round of inputs is guaranteed available. You *can* technically read the first input value inside `ngOnChanges` (checking `firstChange`), but it's the wrong

tool for one-time init: `ngOnChanges` re-fires on every subsequent input change, so any one-time setup code placed there needs an explicit guard, whereas `ngOnInit` already guarantees "exactly once" semantics natively. Also, if the component has no bound inputs at all, `ngOnChanges` never fires, silently skipping your init logic — a real bug magnet.

Q13. What's the correct pattern for unsubscribing from an Observable to avoid memory leaks, and where does it belong?

A: Options, in order of idiomatic preference: (1) use the `async` pipe in the template, which auto-unsubscribes when the view is destroyed — no manual cleanup needed; (2) use `takeUntilDestroyed()` (Angular 16+, injection-context-aware) to automatically complete the subscription when the component is destroyed; (3) manually store the `Subscription` and call `.unsubscribe()` inside `ngOnDestroy`; (4) the `takeUntil(this.destroy$)` pattern, calling `destroy$.next()`; `destroy$.complete()`; in `ngOnDestroy`. The cleanup logic must live in `ngOnDestroy` specifically because it is the only hook guaranteed to run exactly once right before Angular tears down the component — doing it anywhere else either runs too early (component still alive) or not reliably at all.

Q14. Two sibling child components both project content and have view children. Does one child's `ngAfterViewInit` block the other's `ngOnInit`?

A: No — hook execution follows the tree traversal order (Ivy processes `LView`s depth-first, in template declaration order), but each child completes its *entire* lifecycle sequence (constructor through `ngAfterViewChecked`) before Ivy moves to the next sibling in that same parent's child list, given both are created in the same initial CD pass. So for siblings A and B declared in that order, you get: A's full sequence (constructor → ... → `ngAfterViewChecked`) completes entirely, *then* B's full sequence begins — they are not interleaved hook-by-hook. This is a subtlety many candidates get wrong, assuming Angular processes "all `ngOnInits` first, then all `ngAfterViewInits`" breadth-first across the whole tree; it does not — it's depth-first per branch.

7. Quick Revision Cheat Sheet

- **Order (first render):** constructor → `ngOnChanges` → `ngOnInit` → `ngDoCheck` → `ngAfterContentInit` → `ngAfterContentChecked` → `ngAfterViewInit` → `ngAfterViewChecked`.
- **Order (subsequent CD cycles):** `ngOnChanges` (if inputs changed) → `ngDoCheck` → `ngAfterContentChecked` → `ngAfterViewChecked`. (No Init hooks, no constructor.)
- **Destroy:** `ngOnDestroy` only, once, children before parent.
- **Parent/child tree:** top-down for constructor/OnInit/DoCheck; bottom-up for AfterContent/AfterView (children fully finish before parent's AfterView hooks fire). Destroy is also bottom-up (child destroyed before parent).
- **Constructor** = DI only. No inputs, no view, no business logic.
- **ngOnChanges** fires only for `@Input()`-bound properties, only on reference change, never if there are zero inputs.
- **SimpleChange** = `{ previousValue, currentValue, firstChange }`, keyed by public input name.
- **Content vs View:** Content = `<ng-content>` projected from parent (`@ContentChild`), resolved in `ngAfterContentInit`. View = component's own template (`@ViewChild`), resolved in `ngAfterViewInit`.
- **ngDoCheck** = custom dirty-checking; runs every cycle; needed to catch in-place mutation; keep it cheap.
- **ExpressionChangedAfterItHasBeenCheckedError** = mutating a bound value inside AfterView/AfterContent(Checked) hooks; fix by moving logic earlier or forcing an explicit extra CD pass.

- **ngOnDestroy** = only reliable cleanup hook; unsubscribe observables, clear timers/intervals, disconnect observers; not guaranteed for manually created, undetached components.
- **OnPush** changes *whether* a component gets checked, not the *order* of hooks when it is checked.
- **Signal inputs** still trigger `ngonChanges`, but idiomatic reaction to signal changes uses `effect()` / `computed()`, not `ngonChanges`.
- **Ivy internals:** hooks are compiled into flags on `TView`; execution driven by `refreshView` recursing into child `LViews` before firing the parent's `AfterContent/AfterView` hooks; destroy hooks live in a separate `destroyHooks` array invoked bottom-up on view removal.

Created By - Durgesh Singh